

Testing Cheatsheet

Lessons: [18 — Testing & Benchmarking](#) · [40 — Testing Architecture](#) · [49 — Every Kind of Test](#)

Examples: [40](#) · [49](#)

Covers: the `testing` API, every kind of test, `httptest`, fakes vs mocks, run modes

Legend: `[*]` = real Go API that the lessons have not covered yet

FILE & FUNCTION SHAPES

<code>foo_test.go</code>	tests live beside the code they test
<code>package foo</code>	internal test: can reach unexported symbols
<code>package foo_test</code>	external test: only the public API (better default)
<code>func TestXxx(t *testing.T)</code>	a test; the name after Test must be exported-style
<code>func BenchmarkXxx(b *testing.B)</code>	a benchmark
<code>func FuzzXxx(f *testing.F)</code>	a fuzz target
<code>func ExampleXxx()</code>	compiled, run, and shown in the docs
<code>func TestMain(m *testing.M)</code>	one setup/teardown for the whole package
<code>testdata/</code>	fixtures; the go tool ignores this directory

*testing.T

<code>t.Error(args) / t.Errorf</code>	mark failed, KEEP GOING
<code>t.Fatal(args) / t.Fatalf</code>	mark failed, STOP this test now
<code>t.Log / t.Logf</code>	shown with <code>-v</code> , or on failure
<code>t.Skip() / t.Skipf</code>	skip; <code>t.SkipNow()</code> stops immediately
<code>t.Helper()</code>	report failures at the CALLER's line
<code>t.Run("name", func(t *testing.T){...})</code>	a subtest
<code>t.Parallel()</code>	run this test alongside its siblings
<code>t.Cleanup(fn)</code>	LIFO teardown, runs even on failure
<code>t.TempDir()</code>	<code>[*]</code> an auto-removed temp directory
<code>t.Setenv("K", "v")</code>	<code>[*]</code> env var restored after the test (blocks <code>Parallel</code>)
<code>t.Chdir(dir)</code>	<code>[*]</code> Go 1.24+: <code>cd</code> for the duration of the test
<code>t.Context()</code>	<code>[*]</code> Go 1.24+: a <code>ctx</code> cancelled when the test ends
<code>t.Name()</code>	the current test's name
<code>t.Failed()</code>	<code>[*]</code> did it already fail?
<code>t.Deadline()</code>	<code>[*]</code> <code>-> (time, ok)</code> from <code>-timeout</code>

TABLE-DRIVEN TESTS (the default shape)

```
tests := []struct{                                a slice of anonymous structs
    name string; in int; want int
}{
    {"zero", 0, 0},
    {"negative", -3, 3},
}
for _, tt := range tests {                        one loop
    t.Run(tt.name, func(t *testing.T) {           one subtest per case
        if got := Abs(tt.in); got != tt.want {
            t.Errorf("Abs(%d) = %d, want %d", tt.in, got, tt.want)
        }
    })
}
go test -run TestAbs/negative                    run ONE case by name
(name the cases: the name is what you read when CI fails)
```

BENCHMARKS

```
func BenchmarkX(b *testing.B)
for i := 0; i < b.N; i++    the classic loop
for b.Loop()                [*] Go 1.24+: the modern form, immune to elimination
b.ReportAllocs()            include allocs/op in the output
b.ResetTimer()              discard setup time
b.StopTimer() / b.StartTimer() exclude a slow section
b.SetBytes(n)               [*] report throughput in MB/s
b.RunParallel(func(pb *testing.PB){ for pb.Next() {...} }) [*] contention test
b.SetParallelism(p)         [*] goroutines per CPU in RunParallel
go test -bench=. -benchmem  run them, with allocation counts
go test -bench=. -count=10 | benchstat  [*] compare runs honestly
testing.AllocsPerRun(n, f) [*] allocations of one function
(assign the result to a package var or use b.Loop, or it gets optimized away)
```

EXAMPLES (docs that are tested)

```
func ExampleAdd()           documents the Add function
func ExampleUser_Name()    [*] documents a method
func Example()              [*] documents the whole package
// Output:                  everything after it must match stdout exactly
// Unordered output:        [*] compare ignoring line order
no Output comment           compiled but NOT run
(these appear on pkg.go.dev – the only docs that can't rot)
```

FUZZING

```
func FuzzParse(f *testing.F)
f.Add("seed input")         a seed corpus entry
f.Fuzz(func(t *testing.T, s string) { ... }) the target
go test -fuzz=FuzzParse     fuzz until it finds a failure
go test -fuzz=Fuzz -fuzztime=30s  [*] bounded run
testdata/fuzz/              failing inputs are written here, and become tests
(assert INVARIANTS: no panic, round-trip equality, output always valid)
```

PROPERTY-BASED TESTS

```
testing/quick               [*] the stdlib's property checker (frozen, but it works)
quick.Check(f, nil)         [*] f takes generated args, returns bool; 100 random runs
quick.CheckEqual(f, g, nil) [*] two implementations must agree on every input
&quick.Config{MaxCount: 1000} [*] more runs, a custom rand, custom generators
Generator interface         [*] implement Generate(rand, size) for your own types
the mindset                 assert an INVARIANT, not one example:
                             round-trip: Decode(Encode(x)) == x
                             idempotence: f(f(x)) == f(x)
                             oracle: fast(x) == obviouslyCorrect(x)
                             invariant: len(Sort(s)) == len(s), and it is ordered
fuzzing vs property         fuzz explores bytes for crashes; property checks a
                             stated law over generated VALUES
pggregory.net/rapid         [*] the modern third-party choice: shrinking that works
```

httptest

```
httptest.NewRecorder()      an in-memory ResponseWriter
httptest.NewRequest("GET", "/", body)  a request, no network
handler.ServeHTTP(rec, req)  call the handler directly – fast and hermetic
rec.Code / rec.Body / rec.Header()    assert on the result
httptest.NewServer(handler)  a REAL server on a random port; defer srv.Close()
srv.URL                      its base URL, for a real http.Client
httptest.NewTLSServer(h) [*] the same over TLS; srv.Client() trusts it
httptest.NewUnstartedServer(h) [*] configure before Start()
```

TestMain & GOLDEN FILES

```
func TestMain(m *testing.M) { setup(); code := m.Run(); teardown(); os.Exit(code) }
                                package-wide setup (a DB, a container, a fixture)
os.Exit(m.Run())              the minimum body
var update = flag.Bool("update", false, "rewrite golden files")
if *update { os.WriteFile(golden, got, 0o644) }  the golden-file idiom
want, _ := os.ReadFile("testdata/x.golden")      then compare
go test -update                                  [*] regenerate them on purpose
(golden files suit big outputs: rendered HTML, JSON, formatted reports)
```

BUILD TAGS & INTEGRATION TESTS

```
//go:build integration      first line, above package
go test -tags=integration ./...  opt in explicitly
if testing.Short() { t.Skip() }  skip the slow ones under -short
go test -short ./...          the fast subset
//go:build !windows         [*] platform-specific tests
(keep unit tests dependency-free; gate anything that needs infra)
```

RUN MODES & FLAGS

```
go test ./...              every package below here
go test -v                 show every test name and t.Log output
go test -run TestName      regexp over test names; / separates subtests
go test -race              the data-race detector – use it in CI
go test -cover             statement coverage
go test -coverprofile=c.out && go tool cover -html=c.out  the visual report
go test -count=1           defeat the test cache
go test -timeout 30s      [*] panic with all stacks if it hangs
go test -parallel n       [*] max parallel tests
go test -cpu=1,4          [*] run the suite at different GOMAXPROCS
go test -failfast         [*] stop after the first failure
go test -json             [*] machine-readable output for CI
```

FAKES, MOCKS & TEST DOUBLES

fake	a working in-memory implementation (a map repo)
stub	returns canned answers, no behaviour
mock	asserts HOW it was called (order, arity)
spy	records calls, asserts afterwards
prefer FAKES	they survive refactors; mocks test the wiring
define the interface in the CONSUMER	so the fake is tiny
func(...) as the seam	[*] a func field is often simpler than an interface
inject a clock	func() time.Time – never call time.Now() directly
inject the randomness	pass a *rand.Rand or a seed
(if faking is hard, the design is too coupled – that's the real finding)	

TESTING ARCHITECTURE

unit	one package, no I/O, milliseconds
integration	real DB/queue, one service, seconds
contract	the boundary between two services matches
end-to-end	the whole system, few and slow
the pyramid	many unit, some integration, very few e2e
testcontainers	[*] spin up a real Postgres/Redis per test run
one assertion subject	a test that can fail for 5 reasons teaches nothing
determinism	no time.Now, no rand, no network, no sleeps
test behaviour, not methods	the test should survive a refactor
arrange / act / assert	three visible blocks in every test

TRAPS & MEMORIZE

t.Fatal in a goroutine	illegal – use t.Error, or send to a channel
t.Parallel + t.Setenv	panics; they're incompatible by design
subtests capture the loop var	fine since Go 1.22, a bug before it
missing t.Helper()	failures point at your assert func, not the test
the test cache	a "passing" run that never ran; -count=1 to be sure
sleeping to synchronize	flaky by construction; use channels or sync.Mutex
asserting on map order	random by design; sort first
shared state between tests	breaks under -race and -parallel
testing unexported internals	couple the test to today's implementation
100% coverage as a goal	measures lines executed, not behaviour verified
no -race in CI	the one flag that finds bugs you cannot reproduce